SOFTWARE ENGINEERING LAB

Lab-4: VibeCoding LLM Chat History

Fixing and extending a Pygame Tic-Tac-Toe game with an LLM

Student	Vivaan
SRN	PES1UG24AM071
Assigned repository	SETAPESU26/09_tic_tac_toe
Submission repository	github.com/Vivaan-Atharva/SE-LAB → Lab4/Vivaan
LLM used	Claude (Anthropic)
Date	18 October 2026
Outcome	Task 1–4 complete · 1 commit per task · before/after videos

Conversation

PROMPT 1 · STUDENT

I've been assigned **SETAPESU26/09_tic_tac_toe** for Lab 4 (VibeCoding): https://github.com/SETAPESU26/09_tic_tac_toe

The lab requires me to:

1. Run the Python game and record a ~10 second video **before** changing anything.
2. Use an LLM to fix the broken code and add every feature listed in the repo's README, one task at a time.
3. Record another ~10 second video **after** all changes.
4. Push the code to my own lab repo with a **separate commit for each task**, and never raise a PR to SETAPESU26.

Please read the repo and its README, and summarise exactly what is broken and what has to be built. Then let's go step by step.

CLAUDE

I cloned the repo and read the README and every source file. It's a Pygame Tic-Tac-Toe game: you play **X** by clicking, and the computer plays **O** immediately after you using a random-move AI (`game/ai.py`).

Task	Problem / feature	Root cause in the code
1	Diagonal wins are never detected	`check_winner()` in `rules.py` only builds rows and columns
1	A winning move that fills the last cell is reported as "Draw!"	`check_round_end()` checks *board full* **before** checking for a winner
1	Symbols can still be placed after a round ends	`handle_click()` never checks `round_over`
2	Persistent scoreboard (X wins / O wins / draws)	Not implemented; **R** calls `__init__()`, which wipes all state
3	Clicking an occupied cell overwrites it	`handle_click()` writes to the cell without checking it
4	Choose who starts + separate "restart round" / "reset match" controls	Not implemented

There's also a small display issue: the window is 500 px tall and the "X wins!" banner is centred at y = 500, so half of it is cut off.

Plan: one commit for the starter code, one for the before video, one per task (1 to 4), and one for the after video.

PROMPT 2 · STUDENT

My lab repo is **Vivaan-Atharva/SE-LAB**. Inside each lab folder (`Lab1`, `Lab2`, `Lab3`) there is a separate subfolder for each team member, since we share the repo. Lab 4 is individual, so I'll only be doing my own. How should I structure it, and what are the exact setup commands? I'm on macOS.

CLAUDE

I checked your repo: every lab uses `LabN/<Name>/`. Lab 4 should follow the same pattern:

```
Lab4/
└── Vivaan/
    ├── before.mov
    ├── after.mov
    └── tic-tac-toe/        (the updated code)
```

First confirm your tools (`python --version`, `git --version`; Python must be 3.10+). Then clone both repos side by side, copy the starter game in, and commit it untouched:

```
cd ~/Desktop
git clone https://github.com/Vivaan-Atharva/SE-LAB.git
git clone https://github.com/SETAPESU26/09_tic_tac_toe.git
cd SE-LAB
mkdir -p Lab4/Vivaan
cp -r ../09_tic_tac_toe/tic-tac-toe Lab4/Vivaan/
cp ../09_tic_tac_toe/.gitignore Lab4/Vivaan/
git add Lab4
git commit -m "Lab4 (Vivaan): add original tic-tac-toe starter code"
```

PROMPT 3 · STUDENT

Python 3.12.4 and git 2.50.1 are installed. I ran the setup, but I accidentally left the placeholder in, so the folder was created as `Lab4/NAME`:

```
[main 2f61843] Lab4 (NAME): add original tic-tac-toe starter code
 8 files changed, 222 insertions(+)
 create mode 100644 Lab4/NAME/tic-tac-toe/game/game_engine.py
 ...
```

How do I fix this cleanly without adding an extra "fix" commit?

CLAUDE

Since nothing has been pushed yet, you can rename the folder and **amend** the same commit:

```
git mv Lab4/NAME Lab4/Vivaan
git commit --amend -m "Lab4 (Vivaan): add original tic-tac-toe starter code"
```

Then install the dependency and run the original game to see the bugs for yourself:

```
cd Lab4/Vivaan/tic-tac-toe
pip3 install -r requirements.txt
python3 main.py
```

PROMPT 4 · STUDENT

The game window opens. While playing, I completed three X's in a row, but the game didn't stop or announce a winner. Is that one of the bugs?

CLAUDE

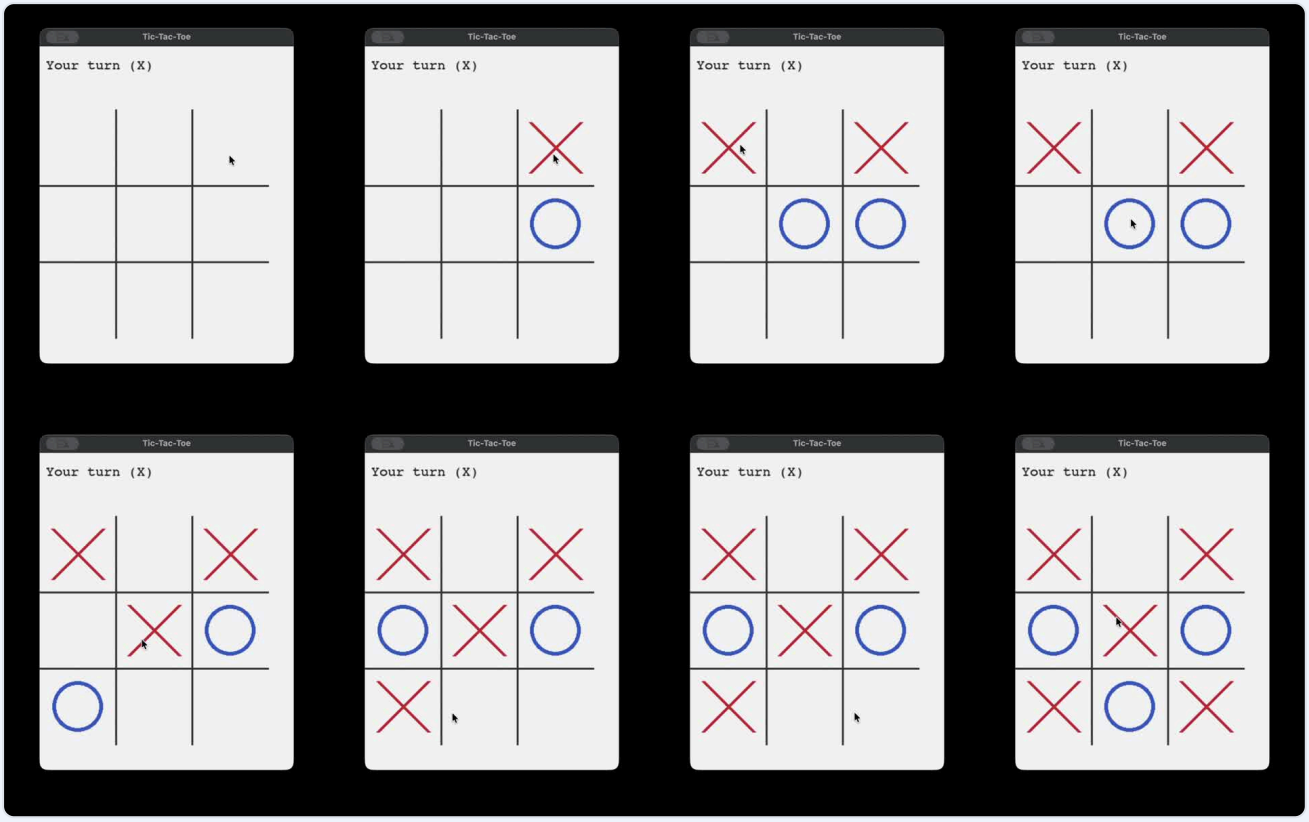
Yes. Your line was almost certainly a **diagonal**. `check_winner()` only checks the 3 rows and 3 columns, so diagonal wins are invisible to the game. This is exactly what the "before" video should show.

Recording the before video (macOS): press **Cmd + Shift + 5**, then choose **Record Selected Portion** and select the game window. Show:

1. **Overwrite bug:** click a cell that already holds an O, and your X replaces it.
2. **Diagonal bug:** complete a diagonal of X's; no winner is announced and play continues.

PROMPT 5 · STUDENT

Here is my before recording (`Screen Recording 2026-10-18 at 2.20.36 PM.mov`). Please verify that it captures the bugs, and give me the exact commands to add it to the repo.



Frames sampled from the before video

CLAUDE

I sampled frames from the video. It clearly shows all three issues: your X **overwrites the centre O**, X completes **both diagonals with no winner announced**, and the label still says **"Your turn (X)"** afterwards, so play continues. The file is about 1 MB, well within GitHub's limits.

```
cd ~/Desktop/SE-LAB
mv ~/Desktop/Screen\ Recording\ 2026-10-18\ at\ 2.20.36*.mov Lab4/Vivaan/before.mov
git add Lab4/Vivaan/before.mov
git commit -m "Lab4 (Vivaan): add before video"
git log --oneline -3
```

PROMPT 6 · STUDENT

```
[main 04696f4] Lab4 (Vivaan): add before video
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 Lab4/Vivaan/before.mov
04696f4 (HEAD -> main) Lab4 (Vivaan): add before video
0c17057 Lab4 (Vivaan): add original tic-tac-toe starter code
4baecb3 (origin/main, origin/HEAD) Add Dhanush Lab 2 and Lab 3 submissions
```

Both commits are in. Let's start **Task 1**: diagonals must count as wins, a real win must be reported as a win even when the board is full, and no moves should be accepted after a round ends. Please keep the change limited to Task 1, since each task needs its own commit.

CLAUDE

Task 1 needs three targeted fixes:

- `rules.py` : add both diagonals to the list of winning lines.
- `game_engine.py` → `check_round_end()` : check for a winner **first**; declare a draw only if there's no winner *and* the board is full.
- `game_engine.py` → `handle_click()` : return immediately when `round_over` is set.

I'm also raising the window height from 500 to 560 px so the "X wins!" banner is actually visible. Occupied-cell overwriting is intentionally left alone, because that's Task 3.

```
cd ~/Desktop/SE-LAB/Lab4/Vivaan/tic-tac-toe

cat > game/rules.py <<'EOF'
"""
rules: win/draw detection for a 3x3 board.

board is a 3x3 list of lists, each cell is None, 'X', or 'O'.
"""


def check_winner(board):
    lines = []
    for r in range(3):
        lines.append([board[r][0], board[r][1], board[r][2]])
    for c in range(3):
        lines.append([board[0][c], board[1][c], board[2][c]])
    # Task 1 fix: both diagonals are winning lines too
    lines.append([board[0][0], board[1][1], board[2][2]])   # top-left -> bottom-right
```

```
    lines.append([board[0][2], board[1][1], board[2][0]])   # top-right -> bottom-left

    for line in lines:
        if line[0] is not None and line[0] == line[1] == line[2]:
            return line[0]
    return None


def is_board_full(board):
    return all(cell is not None for row in board for cell in row)
EOF

cat > game/game_engine.py <<'EOF'
"""
GameEngine: owns the board, turn state, and round-end logic.

You (the player) always play X and click to move. The computer always
plays O and moves automatically right after you, using a simple
random-move AI (see game/ai.py).
"""

from game.rules import check_winner, is_board_full
from game.renderer import board_pos_to_cell
from game.ai import choose_move

HUMAN_SYMBOL = 'X'
COMPUTER_SYMBOL = 'O'


class GameEngine:
    def __init__(self):
        self.board = [[None] * 3 for _ in range(3)]
        self.current_player = 'X'
        self.round_over = False
        self.winner = None   # 'X', 'O', or None (meaning draw, only valid when round_over)

    def handle_click(self, pos):
        if self.round_over:
            return   # Task 1: no moves once the round has ended
        if self.current_player != HUMAN_SYMBOL:
            return   # not your turn - the computer is about to move (or already has)
        cell = board_pos_to_cell(pos)
        if cell is None:
            return
        row, col = cell
        self.board[row][col] = self.current_player   # BUG: doesn't check if the cell is already
occupied
        self.check_round_end()
        self.current_player = 'O' if self.current_player == 'X' else 'X'
```

```
        self._maybe_take_computer_turn()

    def _maybe_take_computer_turn(self):
        if self.round_over or self.current_player != COMPUTER_SYMBOL:
            return
        move = choose_move(self.board)
        if move is None:
            return
        row, col = move
        self.board[row][col] = self.current_player
        self.check_round_end()
        self.current_player = 'O' if self.current_player == 'X' else 'X'

    def handle_keydown(self, key):
        import pygame
        if key == pygame.K_r:
            self.__init__()

    def check_round_end(self):
        # Task 1: look for a winner FIRST, only then check for a draw
        winner = check_winner(self.board)
        if winner:
            self.round_over = True
            self.winner = winner
        elif is_board_full(self.board):
            self.round_over = True
            self.winner = None

    def draw(self, surface, font):
        from game import renderer
        renderer.draw_board(surface, self.board)
        turn_label = "Your turn (X)" if self.current_player == HUMAN_SYMBOL else "Computer's turn
(O)"
        renderer.draw_text(surface, font, turn_label, (10, 20))

        if self.round_over:
            text = f"{self.winner} wins!" if self.winner else "Draw!"
            renderer.draw_banner(surface, font, f"{text} Press R for a new round.")
EOF

sed -i '' 's/WIDTH, HEIGHT = 400, 500/WIDTH, HEIGHT = 400, 560/' game/renderer.py
python3 main.py
```

Test: win on a diagonal and you should see "X wins!". After the win, clicking empty cells should do nothing.

```
cd ~/Desktop/SE-LAB
git add Lab4/Vivaan/tic-tac-toe
git commit -m "Lab4 (Vivaan) Task 1: fix win and draw detection"
```

PROMPT 7 · STUDENT

Tested: diagonal wins are announced and the board locks after a win. Committed:

```
[main 7cc474f] Lab4 (Vivaan) Task 1: fix win and draw detection
 3 files changed, 11 insertions(+), 14 deletions(-)
```

Next, **Task 2**: a scoreboard for X wins, O wins and draws that survives a new round (R) and is cleared only by a full match reset.

CLAUDE

The key design point is to separate **round state** from **match state**. The original R key called `self.__init__()`, which would wipe a scoreboard. So:

- `self.scores` is created once in `__init__`.
- `new_round()` clears only the board and turn.
- `reset_match()` clears the scores and then starts a new round.
- `check_round_end()` increments the right counter, and `draw()` shows the scoreboard at all times.
- **R** maps to `new_round()`, and **M** maps to `reset_match()`.

```
cd ~/Desktop/SE-LAB/Lab4/Vivaan/tic-tac-toe

cat > game/game_engine.py <<'EOF'
"""
GameEngine: owns the board, turn state, round-end logic and the scoreboard.

You (the player) always play X and click to move. The computer always
plays O and moves automatically right after you, using a simple
random-move AI (see game/ai.py).
"""

from game.rules import check_winner, is_board_full
from game.renderer import board_pos_to_cell
from game.ai import choose_move

HUMAN_SYMBOL = 'X'
COMPUTER_SYMBOL = 'O'


class GameEngine:
    def __init__(self):
        # Task 2: the scoreboard is created once and survives new rounds
        self.scores = {'X': 0, 'O': 0, 'Draw': 0}
        self.new_round()
```

```
    def new_round(self):
        """Clear the board only - the scoreboard is kept."""
        self.board = [[None] * 3 for _ in range(3)]
        self.current_player = 'X'
        self.round_over = False
        self.winner = None   # 'X', 'O', or None (meaning draw, only valid when round_over)

    def reset_match(self):
        """Clear the board AND the scoreboard."""
        self.scores = {'X': 0, 'O': 0, 'Draw': 0}
        self.new_round()

    def handle_click(self, pos):
        if self.round_over:
            return   # Task 1: no moves once the round has ended
        if self.current_player != HUMAN_SYMBOL:
            return   # not your turn - the computer is about to move (or already has)
        cell = board_pos_to_cell(pos)
        if cell is None:
            return
        row, col = cell
        self.board[row][col] = self.current_player   # BUG: doesn't check if the cell is already
occupied
        self.check_round_end()
        self.current_player = 'O' if self.current_player == 'X' else 'X'
        self._maybe_take_computer_turn()

    def _maybe_take_computer_turn(self):
        if self.round_over or self.current_player != COMPUTER_SYMBOL:
            return
        move = choose_move(self.board)
        if move is None:
            return
        row, col = move
        self.board[row][col] = self.current_player
        self.check_round_end()
        self.current_player = 'O' if self.current_player == 'X' else 'X'

    def handle_keydown(self, key):
        import pygame
        if key == pygame.K_r:
            self.new_round()      # R: new round, scoreboard kept
        elif key == pygame.K_m:
            self.reset_match()    # M: reset whole match, scoreboard cleared

    def check_round_end(self):
        # Task 1: look for a winner FIRST, only then check for a draw
        winner = check_winner(self.board)
```

```
        if winner:
            self.round_over = True
            self.winner = winner
            self.scores[winner] += 1        # Task 2
        elif is_board_full(self.board):
            self.round_over = True
            self.winner = None
            self.scores['Draw'] += 1        # Task 2

    def draw(self, surface, font):
        from game import renderer
        renderer.draw_board(surface, self.board)
        turn_label = "Your turn (X)" if self.current_player == HUMAN_SYMBOL else "Computer's turn
(O)"
        renderer.draw_text(surface, font, turn_label, (10, 20))
        s = self.scores   # Task 2: scoreboard always visible
        renderer.draw_text(surface, font, f"X: {s['X']}   O: {s['O']}   Draws: {s['Draw']}", (10,
55))

        if self.round_over:
            text = f"{self.winner} wins!" if self.winner else "Draw!"
            renderer.draw_banner(surface, font, f"{text} R: new round  M: reset")
EOF

python3 main.py
```

Test: finish a round and the counter goes up. **R** clears the board but keeps the scores, and **M** resets them to 0.

```
cd ~/Desktop/SE-LAB
git add Lab4/Vivaan/tic-tac-toe
git commit -m "Lab4 (Vivaan) Task 2: add persistent scoreboard"
```

PROMPT 8 · STUDENT

```
[main 4478ece] Lab4 (Vivaan) Task 2: add persistent scoreboard
 1 file changed, 20 insertions(+), 3 deletions(-)
```

Scoreboard works. On to **Task 3**: a click on an already-occupied cell must be rejected entirely. The board and whose turn it is should stay exactly the same, and the computer must not move.

CLAUDE

This is the smallest change of the lab. In `handle_click()`, check the cell **before** writing to it and return early if it's occupied. Because the return happens before the symbol is placed, before the turn flips and before the computer is triggered, the whole state stays untouched.

This command applies exactly that edit and shows the diff:

```
cd ~/Desktop/SE-LAB/Lab4/Vivaan/tic-tac-toe

python3 - <<'EOF'
p = "game/game_engine.py"
s = open(p).read()
old = "        self.board[row][col] = self.current_player   # BUG: doesn't check if the cell is
already occupied\n"
new = ("        if self.board[row][col] is not None:\n"
       "            return   # Task 3: occupied cell - reject, board and turn unchanged\n"
       "        self.board[row][col] = self.current_player\n")
assert old in s, "line not found"
open(p, "w").write(s.replace(old, new))
print("Task 3 change applied")
EOF

git diff
python3 main.py
```

Resulting diff:

```
--- a/Lab4/Vivaan/tic-tac-toe/game/game_engine.py
+++ b/Lab4/Vivaan/tic-tac-toe/game/game_engine.py
@@ -41,7 +41,9 @@ class GameEngine:
         if cell is None:
             return
         row, col = cell
-        self.board[row][col] = self.current_player   # BUG: doesn't check if the cell is already
occupied
+        if self.board[row][col] is not None:
+            return   # Task 3: occupied cell - reject, board and turn unchanged
+        self.board[row][col] = self.current_player
         self.check_round_end()
         self.current_player = 'O' if self.current_player == 'X' else 'X'
         self._maybe_take_computer_turn()
```

PROMPT 9 · STUDENT

```
[main a32594a] Lab4 (Vivaan) Task 3: reject moves on occupied cells
 1 file changed, 3 insertions(+), 1 deletion(-)
```

Clicking on an O or X now does nothing. Last feature, **Task 4**: let the player choose whether X or O starts (if O starts, the computer must take the first move automatically), with two separate controls: restart the round (keep the scoreboard) and reset the match (clear it). The choice should take effect from the next round.

CLAUDE

Design for Task 4:

- `self.starting_player` is stored alongside the scoreboard so it persists across rounds. **1** sets X and **2** sets O; it takes effect on the next **R** or **M**.
- `new_round()` sets `current_player = starting_player`, then calls `_maybe_take_computer_turn()`, so when O starts the computer moves first.
- A shared `_place()` helper handles both human and computer moves. It **doesn't switch turns once the round is over**, so the label can correctly read "Round over".
- The HUD shows the turn, the scoreboard and the "next round starts" choice, plus a controls hint: **R: new round, M: reset match**.

```
cd ~/Desktop/SE-LAB/Lab4/Vivaan/tic-tac-toe

cat > game/game_engine.py <<'EOF'
"""
GameEngine: owns the board, turn state, round-end logic and the scoreboard.

You (the player) always play X and click to move. The computer always
plays O and moves automatically right after you, using a simple
random-move AI (see game/ai.py).

Controls:
    Click  - place X in an empty cell
    R      - restart the current round (scoreboard kept)
    M      - reset the whole match (scoreboard cleared)
    1      - X starts the next round
    2      - O (computer) starts the next round
"""

import pygame

from game.rules import check_winner, is_board_full
from game.renderer import board_pos_to_cell
from game.ai import choose_move

HUMAN_SYMBOL = 'X'
COMPUTER_SYMBOL = 'O'


class GameEngine:
    def __init__(self):
        # Task 2: scoreboard lives outside the per-round state
        self.scores = {'X': 0, 'O': 0, 'Draw': 0}
        # Task 4: who starts each round (takes effect on the next round)
        self.starting_player = 'X'
        self.new_round()

    # ---------- round / match control ----------

    def new_round(self):
        """Clear the board only. Scoreboard and starting choice are kept."""
```

```
        self.board = [[None] * 3 for _ in range(3)]
        self.current_player = self.starting_player
        self.round_over = False
        self.winner = None   # 'X', 'O', or None (draw, only valid when round_over)
        # If O was chosen to start, the computer makes the first move
        self._maybe_take_computer_turn()

    def reset_match(self):
        """Clear the board AND the scoreboard."""
        self.scores = {'X': 0, 'O': 0, 'Draw': 0}
        self.new_round()

    # ---------- input ----------

    def handle_click(self, pos):
        # Task 1 fix (problem 3): no moves once the round has ended
        if self.round_over:
            return
        if self.current_player != HUMAN_SYMBOL:
            return
        cell = board_pos_to_cell(pos)
        if cell is None:
            return
        row, col = cell
        # Task 3 fix: reject clicks on occupied cells - board and turn unchanged
        if self.board[row][col] is not None:
            return
        self._place(row, col)
        self._maybe_take_computer_turn()

    def handle_keydown(self, key):
        if key == pygame.K_r:
            self.new_round()
        elif key == pygame.K_m:
            self.reset_match()
        elif key == pygame.K_1:
            self.starting_player = 'X'
        elif key == pygame.K_2:
            self.starting_player = 'O'

    # ---------- turn logic ----------

    def _place(self, row, col):
        self.board[row][col] = self.current_player
        self.check_round_end()
        if not self.round_over:
            self.current_player = 'O' if self.current_player == 'X' else 'X'

    def _maybe_take_computer_turn(self):
```

```
        if self.round_over or self.current_player != COMPUTER_SYMBOL:
            return
        move = choose_move(self.board)
        if move is None:
            return
        row, col = move
        self._place(row, col)

    def check_round_end(self):
        # Task 1 fix (problem 2): look for a winner BEFORE checking for a full board
        winner = check_winner(self.board)
        if winner:
            self.round_over = True
            self.winner = winner
            self.scores[winner] += 1
        elif is_board_full(self.board):
            self.round_over = True
            self.winner = None
            self.scores['Draw'] += 1

    # ---------- drawing ----------

    def draw(self, surface, font):
        from game import renderer
        renderer.draw_board(surface, self.board)

        if self.round_over:
            turn_label = "Round over"
        elif self.current_player == HUMAN_SYMBOL:
            turn_label = "Your turn (X)"
        else:
            turn_label = "Computer's turn (O)"
        renderer.draw_text(surface, font, turn_label, (10, 8))

        s = self.scores
        renderer.draw_text(surface, font, f"X: {s['X']}   O: {s['O']}   Draws: {s['Draw']}", (10,
38))
        renderer.draw_text(surface, font, f"Next round starts: {self.starting_player} (1=X, 2=O)",
(10, 68),
                           color=renderer.COLOR_HINT)

        if self.round_over:
            text = f"{self.winner} wins!" if self.winner else "Draw!"
            renderer.draw_banner(surface, font, text)
        renderer.draw_hint(surface, font, "R: new round   M: reset match")
EOF

cat > game/renderer.py <<'EOF'
"""
```

```
renderer: all pygame drawing lives here, kept separate from game logic.
"""

import pygame

WIDTH, HEIGHT = 400, 560
BOARD_SIZE = 360
CELL_SIZE = BOARD_SIZE // 3
BOARD_TOP = 100
WINDOW_SIZE = (WIDTH, HEIGHT)

COLOR_BG = (245, 245, 245)
COLOR_LINE = (60, 60, 60)
COLOR_X = (200, 60, 60)
COLOR_O = (60, 100, 200)
COLOR_TEXT = (30, 30, 30)
COLOR_HINT = (110, 110, 110)


def board_pos_to_cell(pos):
    x, y = pos
    y -= BOARD_TOP
    if not (0 <= x < BOARD_SIZE and 0 <= y < BOARD_SIZE):
        return None
    col = x // CELL_SIZE
    row = y // CELL_SIZE
    return int(row), int(col)


def draw_board(surface, board):
    surface.fill(COLOR_BG)
    for i in range(1, 3):
        pygame.draw.line(surface, COLOR_LINE, (i * CELL_SIZE, BOARD_TOP), (i * CELL_SIZE, BOARD_TOP
+ BOARD_SIZE), 3)
        pygame.draw.line(surface, COLOR_LINE, (0, BOARD_TOP + i * CELL_SIZE), (BOARD_SIZE, BOARD_TOP
+ i * CELL_SIZE), 3)

    for r in range(3):
        for c in range(3):
            symbol = board[r][c]
            if symbol is None:
                continue
            center = (c * CELL_SIZE + CELL_SIZE // 2, BOARD_TOP + r * CELL_SIZE + CELL_SIZE // 2)
            if symbol == 'X':
                offset = CELL_SIZE // 3
                pygame.draw.line(surface, COLOR_X, (center[0]-offset, center[1]-offset),
(center[0]+offset, center[1]+offset), 6)
                pygame.draw.line(surface, COLOR_X, (center[0]+offset, center[1]-offset), (center[0]-
offset, center[1]+offset), 6)
```

```
            else:
                pygame.draw.circle(surface, COLOR_O, center, CELL_SIZE // 3, 6)


def draw_text(surface, font, text, pos, color=COLOR_TEXT):
    surface.blit(font.render(text, True, color), pos)


def draw_banner(surface, font, text):
    surf = font.render(text, True, (180, 40, 40))
    rect = surf.get_rect(center=(surface.get_width() // 2, BOARD_TOP + BOARD_SIZE + 30))
    surface.blit(surf, rect)


def draw_hint(surface, font, text):
    surf = font.render(text, True, COLOR_HINT)
    rect = surf.get_rect(center=(surface.get_width() // 2, BOARD_TOP + BOARD_SIZE + 75))
    surface.blit(surf, rect)
EOF

python3 main.py
```

Test: 2 then **R** means an O appears immediately. **1** then **R** gives an empty board. A win shows "Round over" with the score incremented. **R** keeps the scores and **M** clears them.

```
cd ~/Desktop/SE-LAB
git add Lab4/Vivaan/tic-tac-toe
git commit -m "Lab4 (Vivaan) Task 4: first-player choice and separate restart controls"
git log --oneline -7
```

PROMPT 10 · STUDENT

```
[main 03fb0fc] Lab4 (Vivaan) Task 4: first-player choice and separate restart controls
 2 files changed, 73 insertions(+), 28 deletions(-)
03fb0fc (HEAD -> main) Lab4 (Vivaan) Task 4: first-player choice and separate restart controls
a32594a Lab4 (Vivaan) Task 3: reject moves on occupied cells
4478ece Lab4 (Vivaan) Task 2: add persistent scoreboard
7cc474f Lab4 (Vivaan) Task 1: fix win and draw detection
04696f4 Lab4 (Vivaan): add before video
0c17057 Lab4 (Vivaan): add original tic-tac-toe starter code
4baecb3 (origin/main, origin/HEAD) Add Dhanush Lab 2 and Lab 3 submissions
```

All four tasks are committed. What should the after video demonstrate?

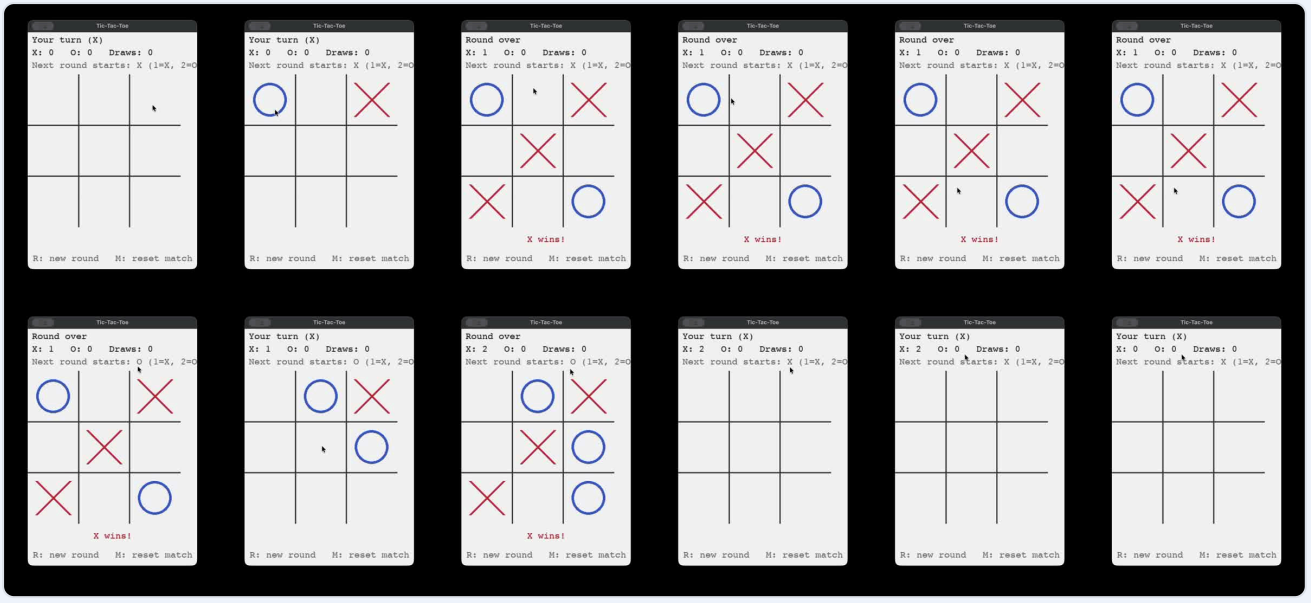
CLAUDE

The history is clean: one commit per task, stacked on top of your team's last commit. In the after video, show:

1. Clicking an O does nothing (Task 3).
2. A diagonal win shows **"X wins!"** and **"Round over"**, the score increments, and further clicks are ignored (Task 1 and Task 2).
3. **2** then **R** makes the computer move first, with the score kept (Task 4).
4. **M** resets the scores to 0 (Task 2 and Task 4).

PROMPT 11 · STUDENT

Here is my after recording (`Screen Recording 2026-10-18 at 2.31.37 PM.mov`). Please check that it demonstrates every fix and feature, and give me the commands to commit and push.



Frames sampled from the after video

CLAUDE

Verified from the sampled frames:

Check	Shown in the video
Diagonal win detected	X completes the top-right to bottom-left diagonal, and **"X wins!"** appears
Round locks after a win	The label reads **"Round over"**; clicks add nothing
Occupied cell rejected	Clicking the O does nothing
Persistent scoreboard	`X: 1` becomes `X: 2` across rounds and is kept after **R**
O starts	"Next round starts: O", and the computer's O is on the board at the start of the round
Match reset	**M** returns the scores to `X: 0 O: 0 Draws: 0`

```
cd ~/Desktop/SE-LAB
mv ~/Desktop/Screen\ Recording\ 2026-10-18\ at\ 2.31.37*.mov Lab4/Vivaan/after.mov
git add Lab4/Vivaan/after.mov
git commit -m "Lab4 (Vivaan): add after video"
```

```
git pull --rebase origin main
git push origin main
```

PROMPT 12 · STUDENT

Pushed. Can you check the repo on GitHub and confirm everything is correct?

CLAUDE

I cloned **Vivaan-Atharva/SE-LAB** fresh and checked it:

- **History:** separate commits, in order: starter code, before video, **Task 1**, **Task 2**, **Task 3**, **Task 4**, after video. Lab 1 to Lab 3 and your teammates' folders are untouched.
- **Files:** `Lab4/Vivaan/` contains `before.mov`, `after.mov` and `tic-tac-toe/` with all game files. No cache files were committed.
- **Code:** the pushed code matches the tested version exactly, and passes automated checks for all four tasks (diagonal wins, win-on-last-cell, board lock, occupied-cell rejection, scoreboard persistence and reset, O-starts).
- **No pull request** was opened to SETAPESU26.